

## Era Boosting

### Performance stationarity

Two models predict market direction 60% of the time, they can have same returns but carry completely different risk.

Model A: TTTFTTFTTF

Model B: TTTFFFFTTTTTTT ( $\Delta$  long burns)

Have model A is better because its sequence behaves with more stationarity. (lacks autocorrelation / more memoryless / no long burns)

### Non-stationary features

One possible root cause for non-stationary returns, is non-stationary features. The features can work well in specific regimes and then cycle losses in next regimes.

Parallel with feature neutralization where we rely too much on certain features.

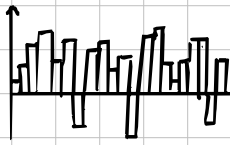
### Optimizing for correlation vs. stationarity

The Sharpe ratio already penalizes variance of returns, but it treats scattered losses, the same as big series of losses.

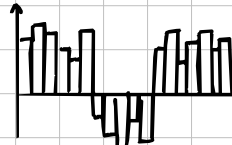
That's why we can introduce smart Sharpe ratio which also penalizes for autocorrelation.

$$\text{Sharpe} = \frac{\text{mean returns}}{\text{std returns}}$$

$$\text{vs. smart Sharpe} = \frac{\text{mean returns}}{\text{std } r \times \text{autocorr penalty}}$$



model A



model B

Sharpe A = Sharpe B

Smart Sharpe A > B

### Implementation

Train model at each iteration only with eras having worst correlations, and growing ensemble this way.

This way model is forced to reduce clumps of losses by creating new trees to address low corrs.

```
function era_boost (X, y, proportion, trees_per_step, num_iters)
    model_fit (X, y, n_estimators = trees_per_step) # on ALL eras
    for n in num_iters:
        predictions = model.predict(X)
        era_corr = spearman_corr(predictions, y)
        worst_eras = sort(era_corrs)[: len(era_corrs) * proportion]
        model.n_estimators += trees_per_step
        current_ensemble = model.get_booster()
        model_fit (worst_eras[X], worst_eras[y], current_ensemble)
```